

Python 3.12: New Features, Performance, and Impact – An Academic Overview

Abstract

Python 3.12, released in October 2023, is a significant update that introduces new language syntax, performance optimizations, standard library enhancements, and removes outdated features. This paper provides a comprehensive overview of Python 3.12's improvements, including its new type parameter syntax (PEP 695) and enhanced f-string grammar (PEP 701), interpreter changes enabling per-interpreter GIL (PEP 684) and low-overhead monitoring (PEP 669), and numerous standard library updates. We compare Python 3.12 to Python 3.11 and earlier versions, highlighting performance gains (roughly 5% overall) and continued progress in error messages and developer experience. Deprecations and removals (e.g. elimination of `distutils` and older modules) are also documented. Finally, we discuss real-world implications for developers, students, and researchers adopting Python 3.12. The analysis draws on official Python Enhancement Proposals (PEPs), documentation, and authoritative sources, presenting the findings in a structured scholarly format.

Introduction

Python 3.12 is the latest major release in the Python 3.x series, following Python 3.11 and continuing the language's evolution in terms of usability, performance, and maintainability. Python 3.12.0 was officially released on October 2, 2023 ¹. This release arrives roughly one year after Python 3.11, in line with Python's annual release cadence (as per PEP 602). It contains a mix of language changes, interpreter optimizations, standard library additions, and the removal of deprecated "dead" batteries. The focus of Python 3.12's changes can be summarized in two broad themes: (1) **Enhanced developer experience and language expressiveness**, through new syntax features and more helpful error messages; and (2) **Improved performance and modernization**, via interpreter optimizations and cleanup of legacy components ² ³. Notably, Python 3.12 introduces a new syntax for type parameters in generics and type aliases (simplifying static typing), lifts several restrictions in f-string literals, and adds support for isolated subinterpreters with separate Global Interpreter Locks (GILs) ⁴. Many standard library modules receive upgrades for better performance or usability, while long-deprecated modules like `asyncore`, `asynchat`, and `distutils` have been removed ⁵ ⁶. In the following sections, we delve into the major new language features, performance improvements, deprecations/removals, library changes, and the practical impact of Python 3.12. We also compare Python 3.12 against its predecessor (3.11) and earlier versions to contextualize its advancements.

New Language Features and Syntax in Python 3.12

Python 3.12 brings several significant language enhancements that improve code clarity and expressiveness. Two of the most prominent changes are the introduction of a **dedicated syntax for type parameters in generics** (PEP 695) and the **formalization of f-string grammar** (PEP 701). These, along with

other improvements to the interpreter and language semantics, aim to make Python more convenient for developers.

- **Type Parameter Syntax (PEP 695)** – PEP 695 introduces a concise, explicit syntax for declaring **generic classes and functions**, replacing the older verbose notation that relied on `TypeVar` declarations ⁷ ⁸. In Python 3.12, one can now declare type parameters directly in the function or class definition using bracket notation. For example, a generic function can be defined as `def max[T](args: Iterable[T]) -> T: ...` and a generic class as `class list[T]: ...` ⁹. This new syntax makes the scope of type variables clear (they are scoped to the function or class where they're defined) and no longer requires separately declaring variance. Additionally, **type aliases** can now be defined using the new `type` statement, introduced by PEP 695, which creates an alias in a single concise declaration ¹⁰. For instance, one can write `type Point = tuple[float, float]` to alias a coordinate pair type, or even define generic type aliases like `type Result[T] = tuple[T, int]` ¹¹. The new syntax supports advanced typing features such as variadic generics: one can include `TypeVarTuple` and `ParamSpec` parameters in the type parameter list, and specify bounds or constraints on type variables within the brackets ⁸. Under the hood, Python 3.12 implements a new **annotation scope** to ensure these type parameters behave properly (they are available inside the generic class or function, including nested scopes, but not leaked outside) ¹². This change greatly improves the ergonomics of static typing – complex generic data structures and functions can be expressed more naturally, aiding both readability and tool support ⁷ ¹³. (PEP 695 was authored by Eric Traut, with implementation by Jelle Zijlstra and others ¹⁴.)

- **Formalized F-String Grammar (PEP 701)** – Python 3.12 removes many of the historical restrictions on f-string literals by integrating their grammar into the core Python parser ¹⁵. PEP 701 defines f-strings in the formal grammar (where previously they were handled by ad-hoc tokenization rules) and thereby allows **any valid Python expression** inside an f-string placeholder. In practical terms, this means that f-string expressions can now include content that was illegal in 3.11: for example, *reuse of the same quote characters* as the containing string, multi-line expressions and comments inside the `{...}` braces, and even backslashes or Unicode escape sequences in the expression ¹⁵. All of these are supported in Python 3.12, whereas in Python 3.11 such usage would result in syntax errors. An illustration of the new capability is that one can nest f-strings arbitrarily deep or include quotes without escaping. For instance, the following is now valid in 3.12: `f"This is the playlist: {', '.join(songs)}"`, where the inner expression uses double quotes even though the outer f-string is in double quotes ¹⁶. Likewise, an f-string can span multiple lines with internal comments for clarity ¹⁷ – improving readability of complex formatted strings (such as SQL queries or JSON templates constructed with f-strings). By leveraging the new PEG parser (introduced in 3.9/3.10 via PEP 617), f-string parsing in 3.12 is not only more powerful but also produces **more precise error messages** when there is a syntax mistake ¹⁸ ¹⁹. In Python 3.11, an error inside an f-string might produce a generic “invalid syntax” pointing at an approximate location, whereas in Python 3.12 the parser can pinpoint the exact position of the error within the f-string and provide a clearer message ²⁰. Overall, PEP 701's changes make formatted string literals more flexible and maintainable for developers, aligning Python with the capabilities of f-string-like interpolations in other languages ²¹ ²².

- **Per-Interpreter GIL (PEP 684)** – Although not a syntax change per se, Python 3.12 introduces a major change to the interpreter's **concurrency model**: it supports creating multiple *isolated*

subinterpreters, each with its own Global Interpreter Lock (GIL) ²³. In current CPython, the GIL is a process-wide lock that prevents true parallel execution of Python bytecode in multiple threads. PEP 684 provides a way to have separate GILs on a per-interpreter basis, which means if an application uses the C-API to spin up independent interpreters within one process, those interpreters can run in parallel on multiple CPU cores ²⁴. This is a stepping stone towards better multi-core Python utilization. In Python 3.12, this feature is **only accessible via the C API** (using the new function `Py_NewInterpreterFromConfig()` with a flag for an own GIL) ²⁵, and there is not yet a high-level Python API to launch subinterpreters with isolated GILs (a Python-level API is anticipated in 3.13) ²⁴. Nonetheless, this change is significant for embedding scenarios and future concurrency frameworks – it lays groundwork for potentially mitigating the GIL bottleneck by allowing parallelism across subinterpreters ²³. (Contributed by Eric Snow and others in CPython's implementation.)

- **Low-Impact Monitoring API (PEP 669)** – Python 3.12 adds a new *monitoring API* designed for tools like profilers, debuggers, coverage analyzers, and other runtime event monitors ²⁶. Prior to 3.12, hooking into Python's execution (via `sys.settrace` or `sys.setprofile`) could incur significant overhead, as the tracing function would be invoked for every line or function event. PEP 669 defines a finer-grained and lower-overhead mechanism to monitor various events (function calls, returns, line execution, exceptions, bytecode jumps, etc.) with an opt-in model ²⁷. In essence, tools can subscribe only to the specific events they need, and “you only pay for what you use” ²⁷. This allows near-zero overhead when monitors are disabled, and much lower overhead when monitoring a limited subset of events, compared to the blanket overhead of previous tracing. The new API is accessible through the `sys.monitoring` module (e.g., one can enable event hooks at the interpreter level) ²⁷. This enhancement is expected to improve the performance and accuracy of development tools; for example, profilers can collect timing information with minimal impact on program execution, and coverage tools can track executed lines more efficiently. Mark Shannon (the author of PEP 669) contributed the implementation to CPython ²⁶.
- **Buffer Protocol Support in Python (PEP 688)** – In Python 3.12, the long-standing C-level “buffer protocol” (which underlies efficient sharing of memory between objects like bytes, bytearray, memoryview, NumPy arrays, etc.) becomes directly accessible to Python classes ²⁸. PEP 688 introduces the `__buffer__()` special method: if a class implements `__buffer__`, its instances can expose a binary buffer interface to other Python or C code ²⁹. The PEP also provides a new abstract base class `collections.abc.Buffer` to mark classes that support the buffer protocol, which can be used in type annotations to signify “buffer-like” objects ³⁰. Additionally, an `inspect.BufferFlags` enum is added to represent flags that control how the buffer is exported ³⁰. This change means that pure Python classes can now behave like built-in bytes or arrays with respect to memory sharing – for instance, a custom data structure could allow `memoryview(obj)` if it defines `__buffer__`. It opens the door for Python-level implementations of efficient I/O or numeric data containers without writing C extensions. Essentially, PEP 688 makes the buffer protocol a first-class Python feature, improving the language's ability to handle binary data and interact with low-level APIs ²⁸.
- **Other Language Improvements** – Python 3.12 includes numerous smaller changes to the language semantics and built-in behaviors. For example, **PEP 709 (Comprehension Inlining)** is an internal optimization that nonetheless has some user-visible effects: list, set, and dict comprehensions no longer create a hidden function frame for the comprehension and are instead inlined into the

surrounding code, yielding up to a 2× speedup in comprehension execution ³¹ ³². Inlined comprehensions still have isolated iteration variables (to avoid leaking variables to the outer scope), but some subtle changes occur (e.g. comprehensions won't appear as separate frames in tracebacks, and `locals()` within a comprehension behaves differently) ³³. Another change is that **error messages continue to improve** beyond syntax errors: Python 3.12's interpreter will now suggest module imports in NameError messages. If a NameError occurs for a name that matches a standard library module, the exception asks "Did you forget to import 'sys'?" (for example) ³⁴. This behavior helps newcomers by pointing out missing imports for standard modules. Similarly, NameError suggestions have been refined in class instance contexts – if you use an undefined name inside a method that matches an attribute set on `self`, Python 3.12 will suggest using `self.name` ³⁵. ImportError messages from failed `from X import Y` now suggest perhaps the correct name available in module X (e.g., mistyping `chainmap` will suggest `ChainMap` from `collections`) ³⁶. These "Did you mean ..." suggestions, first introduced in Python 3.10 and 3.11, are expanded in 3.12 to catch more common mistakes, further enhancing Python's friendliness ³⁷. Other minor language changes include: a new warning (`SyntaxWarning`) when invalid escape sequences are used in string literals (preparing to make them errors in the future) ³⁹, allowing the walrus operator (`:=`) on a loop variable if that variable is not the target of the comprehension (resolving a quirk with assignment expressions in comprehensions) ⁴⁰, making slice objects hashable (so they can be used as dictionary keys) ⁴¹, and improvements to built-ins like `sum()` which now uses Neumaier summation for better floating-point accuracy ⁴². Many of these changes continue Python's trajectory of improving consistency and developer experience in the core language.

Performance Improvements Compared to Previous Versions

One of the key goals of Python 3.12 was to build upon the substantial performance gains delivered in Python 3.11. While Python 3.11 introduced the **Specializing Adaptive Interpreter** (PEP 659) which could make programs *10–60% faster* than 3.10 in various benchmarks ⁴³, Python 3.12 continues this trajectory with targeted optimizations and a few new techniques. The net result is an estimated **~5% overall speed increase** over Python 3.11 in general use ⁴⁴, alongside larger boosts for specific operations. Below, we outline the major performance improvements in Python 3.12:

- **Bytecode and Interpreter Optimizations** – CPython 3.12 includes several bytecode-level changes aimed at speeding up execution. PEP 709 (mentioned above) inlines list/set/dict comprehensions, eliminating function call overhead and giving up to a 2× speedup for comprehension-heavy code ⁴⁵. New bytecode instructions have been introduced or refined: for example, a specialized `LOAD_SUPER_ATTR` instruction was added to optimize attribute access via `super()` calls ⁴⁶ ⁴⁷. This makes method calls on super classes faster than before. The interpreter's internal instruction set was streamlined by removing some redundant opcodes (like `LOAD_METHOD`) and merging their functionality into other opcodes (such as using a flag on `LOAD_ATTR` for method calls) ⁴⁸. Additionally, *BOLT*, a binary optimizer for post-link optimization, is now experimentally supported in the build process – using BOLT can yield a **1–5% speed improvement** by optimizing the machine code layout ⁴⁹. These changes reflect a continued effort to squeeze more performance out of CPython's evaluation loop.
- **Faster Built-ins and Operations** – Several commonly used operations have been accelerated in 3.12. **Regular expression substitutions** (`re.sub` and `re.subn`) are now **2–3× faster** when using

a replacement string with backreference groups, thanks to an optimized implementation ⁵⁰. Creation of `asyncio.Task` objects was optimized by deferring internal string formatting, resulting in faster task scheduling in asynchronous programs ⁵¹. The `tokenize` module benefitted indirectly from PEP 701 (f-string parsing changes) – generating tokens for source code is **up to 64% faster** due to the new approach to f-strings ⁵². This speeds up any tooling that relies on the tokenizer. Furthermore, `isinstance()` checks against `typing.Protocol` **classes (runtime-checkable protocols)** have been vastly sped up: most such checks are at least 2× faster, and some can be **20× faster** than in 3.11 ⁵³. This improvement is particularly relevant for projects using Python’s structural subtyping (protocols) at runtime. Even built-in math operations saw attention: the built-in `sum()` now uses a more numerically stable summation algorithm (Neumaier’s method), which, while aimed at accuracy, also has performance characteristics beneficial for certain input distributions ⁴².

- **Async IO and Concurrency Performance** – The `asyncio` library in Python 3.12 received significant performance tuning. Writing to network sockets with `asyncio` is now more efficient: unnecessary data copies were eliminated and on platforms supporting it, `sendmsg()` is used internally, greatly improving throughput for socket writes ⁵⁴. Additionally, a new *eager task scheduling* option is introduced: developers can opt an event loop into “eager task execution” via `asyncio.eager_task_factory()`, which can make certain patterns of task scheduling **2-5× faster** ⁵⁵. There’s also a new C implementation of `asyncio.current_task()` which gives a 4×-6× speedup for retrieving the current task ⁵⁶. Some high-level benchmarks have shown that these optimizations result in up to a **75% improvement in throughput** for I/O-heavy asyncio programs ⁵⁷. This means Python 3.12 can handle more concurrent connections or higher message volumes using asyncio compared to previous versions, making it attractive for network servers and web frameworks.

- **Memory and Internal Efficiency** – Python 3.12 made internal changes that reduce memory footprint and prepare for future performance gains. One low-level change was removing the deprecated `wstr` and `wstr_length` members from Unicode string objects (PEP 623), which saves 8-16 bytes per string object on 64-bit systems ⁵⁸. While invisible to Python code, this reduces memory overhead and can marginally improve cache performance when many strings are used. Another internal update involves the representation of integers: Python 3.12 changed how int objects are stored, as noted in the release notes, in preparation for future optimizations ⁵⁹. (This change should not affect Python semantics, but it’s mentioned for projects like Cython that interface with the internals.) Additionally, **PEP 683 (Immortal Objects)** was implemented: certain singletons and static objects can be marked “immortal” (no longer tracked by reference counting in the usual way), which avoids some reference management overhead ⁶⁰. This primarily impacts C-API usage and long-lived global objects, improving their performance by skipping refcount checks (the full benefit of immortal objects will be realized in CPython internals over time).

In summary, Python 3.12 delivers moderate but meaningful speed improvements over 3.11. Empirical measurements by the core team estimate about a 5% overall performance boost for the average workload ⁴⁴. Specific tasks can see larger gains: e.g., comprehension-heavy code runs up to twice as fast due to PEP 709 ⁴⁵, and protocol-heavy `isinstance` checks in frameworks like `typing` or `dataclasses` can be an order of magnitude faster ⁵³. These optimizations, combined with the improved concurrency via subinterpreters (PEP 684) and better profiling tools (PEP 669), continue to narrow the performance gap and make Python more efficient for both CPU-bound and I/O-bound use cases.

Deprecations and Removals of Old Features

Consistent with Python's gradual improvement policy, version 3.12 deprecates or removes several outdated features. The emphasis in this release is on cleaning up modules and APIs that have been long deprecated or superseded, thereby streamlining the standard library and reducing maintenance burden ². Notably, Python 3.12 implements parts of **PEP 594**, "Removing Dead Batteries," and **PEP 632**, which targeted the deprecated `distutils` module. Below is a summary of the key deprecations and removals:

- **Removal of Obsolete Modules** – Python 3.12 finally **removes the `distutils` module** from the standard library ⁶¹. `distutils` had been deprecated since Python 3.10 (per PEP 632) in favor of the standalone `setuptools` package. In 3.12, any attempt to import `distutils` will fail by default (though `setuptools` bundles its own copy to maintain build compatibility) ⁶¹. Additionally, the old networking modules `asyncore` and `asynchat` (and the related `smtplib` module) have been removed ⁶ ⁶². These modules date back to early Python 2 and were effectively superseded by `asyncio` and third-party libraries. They were deprecated for years and are now gone as part of PEP 594's effort to eliminate outdated "batteries." Similarly, the old `imp` module (for import machinery) has been removed ⁶; since Python 3.4, `imp` was replaced by `importlib` and marked deprecated. Python 3.12's removal of `imp` means any legacy code still using it must migrate to `importlib` (which provides equivalent functionality). These removals may cause `ImportError` in projects that haven't updated, but modern code should not be affected, as these modules were long discouraged.
- **Deprecated APIs in `unittest`** – A number of long-deprecated names in the `unittest` module have been removed in Python 3.12 ⁶³. This includes several aliases for methods in `unittest.TestCase` that were deprecated as far back as Python 3.1/3.2. For example, aliases like `failUnlessEqual` (older name for `assertEqual`) and similar have been excised ⁶. The removal tidies up the `unittest` API, which now only supports the modern method names. Code bases that for some reason still used the old names will need to update to the standard names (which they likely have, given the deprecation period exceeded a decade).
- **C API and Internal Cleanup** – As mentioned, the internal `wstr` fields in Unicode objects were removed in 3.12 as per PEP 623 ⁵⁸. This is a C-level change (affecting extensions that peered into Unicode internals). Python 3.12 also finalizes the removal of certain legacy C-API functions that were deprecated. For instance, the old `Py_UNICODE` type and related APIs (deprecated since PEP 393's flexible string representation) are fully removed. Another internal change is the modification of the integer object layout for future optimizations ⁵⁹ – while not a runtime removal, it's a deprecation of the assumption about the old layout, potentially affecting C extensions like Cython if they relied on those details.
- **Syntax and Behavior Deprecations** – Python 3.12 escalates the visibility of some misuses that were previously only lightly warned. Notably, using invalid escape sequences in string literals (such as `"\d"` in regular expressions without making it raw) now raises a `SyntaxWarning` instead of a `DeprecationWarning` ³⁹. This means developers are more likely to see the warning (`DeprecationWarnings` are ignored by default, whereas `SyntaxWarnings` appear by default), nudging them to fix these strings. In future Python (expected by 3.14), such invalid escapes will become outright `SyntaxError`s ⁶⁴. Similarly, octal escape sequences greater than `0o377` now trigger a

SyntaxWarning (they were deprecated in 3.11) ⁶⁵. These changes are part of the path to making the language more consistent and preventing common mistakes (like `\d` unintentionally being a normal `'d'` character).

- **Future Removals Announced** – While not removed in 3.12, the release notes indicate features that are now deprecated and slated for removal in coming versions. For example, in the `asyncio` module, the *per-process child watcher* classes (`asyncio.SafeChildWatcher`, etc.) are deprecated in 3.12 and will be removed in 3.14 ⁶⁶. The old `asyncio` child watcher API is being phased out in favor of better cross-platform defaults (PIDFD child watchers on Linux, for instance). Another deprecation is the use of certain legacy AST constants (`ast.Num`, `ast.Str`, etc.), which now raises a `DeprecationWarning` on use, since one should use `ast.Constant` instead ⁶⁷. Python 3.12's documentation explicitly lists many such deprecations (e.g., `argparse.BooleanOptionalAction` had some parameters deprecated ⁶⁸). These notices serve as warnings to developers to update their code before Python 3.14 or 3.15, when the deprecated features will be removed.

In summary, Python 3.12's deprecation and removal actions reinforce the language's commitment to backward-incompatible cleanup at a measured pace. Removing `distutils` and old `async` modules declutters the standard library (in line with PEP 594's plan to retire outdated modules ⁶¹), and deprecating unsafe or obsolete APIs now will lead to a more robust core in future versions. Most everyday users of Python are unlikely to be affected by these removals (as the modules and names in question have had modern replacements for years), but it underscores the importance of keeping code updated with current best practices.

Changes to the Standard Library

Python's standard library in 3.12 sees numerous improvements, both major and minor, across a variety of modules. The Python 3.12 "What's New" documentation emphasizes that library changes focused on **usability, correctness, and performance**, often cleaning up deprecated APIs and adding new conveniences ². In this section, we highlight some of the notable standard library changes in Python 3.12:

- **Filesystem and OS Utilities:** The `pathlib` module has been significantly enhanced. **Subclassing of `pathlib.Path` is now supported** ⁶⁹, meaning users can create their own `Path` subclasses (perhaps to add methods or maintain state) which was not reliably possible before. Along with this, `pathlib.PurePath.with_segments()` can be overridden by subclasses to customize how new path objects are constructed ⁷⁰. A major addition is `pathlib.Path.walk()`, a method that behaves similarly to `os.walk()` for iterating through directory trees ⁷¹. This provides an object-oriented way to traverse files and directories recursively, which improves the ergonomics of file system operations ⁷¹. Other `pathlib` improvements include `Path.is_junction()` (on Windows, to detect directory junctions) ⁷² and an optional `case_sensitive` flag for `Path.glob()` and related methods to control case-sensitivity in matching ⁷³ – useful on case-insensitive file systems. Meanwhile, the `os` module received **better Windows support** ⁷⁴. For instance, new functions `os.listdirives()`, `os.listvolumes()`, and `os.listmounts()` were added on Windows to easily enumerate drive letters, volume GUIDs, and mount points ⁷⁵. The `os.stat()` result on Windows is now more accurate: it populates `st_birthtime` (file creation time) properly, and prepares to change `st_ctime` to mean "last metadata change" (matching Unix

semantics) in the future ⁷⁶. Also, `os.path` got a new `os.path.splitroot()` function which splits a path into (drive, root, rest) components ⁷⁷, and `os.path.isjunction()` to complement the junction detection mentioned above ⁷⁸ ⁷⁹. These enhancements improve cross-platform behavior and offer new tools for system programming in Python.

- **New Utilities in Itertools and Others:** Python 3.12 adds some “battery-included” functions that were previously available only via recipes or external libraries. A notable example is `itertools.batched()`, which groups items from an iterable into batches of a fixed size (the last batch may be smaller) ⁸⁰. This function provides a memory-efficient way to iterate in chunks, often useful for processing or pagination of large data sets. In the `math` module, a new function `math.sumprod()` computes the sum of products of two sequences elementwise ⁸¹. This can be seen as a dot-product utility (though it sums products of corresponding elements; it’s effectively `sum(a[i]*b[i] for i in range(n))`). `math.sumprod()` can improve both the clarity and performance of such calculations by using a dedicated C implementation. The `random` module introduced `random.binomialvariate(n, p)` to generate binomially-distributed random numbers ⁸², expanding the repertoire of distributions available out-of-the-box.

- **Typing and Code Clarity:** Beyond the core language syntax for typing (PEP 695 discussed earlier), the standard library’s typing support improved. Python 3.12 includes **PEP 698 – the `typing.override` decorator** ⁸³. This new decorator can be applied to methods in a subclass to indicate that the method is intended to override a method in the base class ⁸³. Type checkers (like mypy or pyright) will use this to catch errors where a supposed override doesn’t actually match any base class method (e.g., due to a typo in the method name) ⁸⁴. This helps maintain correctness in large codebases by making override intentions explicit. Another typing PEP, **PEP 692**, was implemented: it allows using `TypedDict` for more precise typing of `**kwargs` parameters ⁸⁵. By introducing the `Unpack` operator for `TypedDict`, functions can specify a `TypedDict` type for their keyword arguments, enabling static analysis to catch incorrect keyword usage and document expected keys clearly ⁸⁶. Together, these enhancements further Python’s static typing capabilities, making it easier to write type-safe code.

- **Standard Library Module Improvements:** Many modules have received minor but useful improvements:

- The `asyncio` module, in addition to performance boosts, now defaults to using `asyncio.PidfdChildWatcher` on Linux (if available) for subprocess monitoring ⁸⁷, which is more efficient and avoids problems with `fork`. Also, `asyncio`’s high-level APIs got quality-of-life additions such as `loop_factory` parameter in `asyncio.run()` to allow custom event loop implementations ⁸⁸.
- The `calendar` module now provides convenient enumerations for months and weekdays: `calendar.Month` and `calendar.Day` are `Enum` classes listing months January–December and days Monday–Sunday, respectively ⁸⁹. This addition makes code working with dates more readable (e.g., using `calendar.Month.MARCH` instead of an integer 3).
- In `csv`, two new quoting constants, `csv.QUOTE_NOTNULL` and `csv.QUOTE_STRINGS`, were added to give finer control over how empty strings and `None` are quoted when writing CSV files

⁹⁰.

- The `inspect` module gained `inspect.getasynctgenstate()` and `inspect.getasynctgenlocals()` to inspect asynchronous generators' state ⁹¹. It also now includes `inspect.markcoroutinefunction()` to explicitly mark a regular function as coroutine-producing (for frameworks that use sync functions returning coroutine objects) ⁹¹. Notably, `inspect.getattr_static()` was optimized to be about 2× faster ⁹², which can help introspection-heavy code.
- The `platform` module can now recognize Windows 11 and newer Windows Server versions properly (previously they might be misidentified as Windows 10) ⁹³.
- The `pdb` debugger added convenience variables for use in interactive debugging sessions (to quickly access certain internal values) ⁹⁴.
- The `shutil` module made the archive creation and extraction safer and more flexible: for instance, `shutil.unpack_archive()` now accepts a *filter* argument to restrict tarfile features for security (related to PEP 706) ⁹⁵. Also, `shutil.rmtree()` gained an `onexc` callback that uses an exception instance (improving on the older `onerror`) ⁹⁶.
- `sqlite3` now has a **command-line interface** when invoked with `python -m sqlite3` ⁹⁷ ⁹⁸. This allows basic execution of SQL queries from the command line, similar to the standalone `sqlite3` shell – a handy addition for quick database inspections.
- The `uuid` module similarly got a command-line interface (via `python -m uuid`) to generate UUIDs from the command line ⁹⁹.
- **Security:** Python 3.12 upgraded some of its cryptographic implementations – the `hashlib` module's built-in fallback implementations for algorithms like SHA-2, SHA-3, and MD5 are now derived from the HACL* project's formally verified C code ¹⁰⁰. While OpenSSL is still used by default for these hashes, the built-in ones (used if OpenSSL is unavailable) are now more robust and perhaps more performant due to being formally verified.

It's worth noting that **no new builtin modules** were added in Python 3.12 (the "New Modules" section of the release notes is empty ¹⁰¹), reflecting a focus on improving existing modules rather than expanding the `stdlib`'s scope. The changes above, collectively, polish the standard library: they add functionality that fills gaps (enums in `calendar`, batching in `itertools`), remove antiquated parts (old modules and APIs), and improve cross-platform behavior and performance. For developers, this means fewer external dependencies for common tasks (since the `stdlib` is more capable) and a smoother experience using Python's out-of-the-box tools.

Real-World Use Cases and Practical Impact on Developers

The new features and improvements in Python 3.12 are not just abstract changes – they have concrete implications for various segments of the Python community. In this section, we analyze how Python 3.12's changes benefit developers in practice and provide examples of use cases:

- **Simplified Generic Programming:** The introduction of PEP 695's type parameter syntax directly benefits library authors and projects heavily using **static typing**. For example, consider a library implementing data container classes (like a generic tree or graph structure). In Python 3.11 and earlier, the developer had to declare type variables upfront and use the `Generic[...]` base class, leading to boilerplate and potential confusion about scope. In Python 3.12, one can define `class Tree[T]: ...` with methods referencing the type parameter `T` naturally within the class ⁹. This not only makes the code more readable (mirroring how generics are expressed in languages like Java or C++), but it also reduces errors, since the type parameter's scope is clearly the class itself ¹².

Static type checkers will enforce proper usage, and the new `type` alias statement can help in breaking down complex types for clarity. **Use case:** A data science project can define a `Matrix[T]` class to parameterize matrices by element type (float, Decimal, etc.) and easily declare operations like `def scale(self, factor: T) -> Matrix[T]` without additional typing clutter. This lowers the barrier for teams to adopt Python's type hints in large codebases, leading to more maintainable code.

- **More Powerful f-Strings for Clean Code:** Many Python developers generate dynamic strings – for instance, creating HTML/SQL snippets or logging messages – and often ran into limitations with f-strings in the past. Python 3.12's f-string enhancements (PEP 701) allow such string construction to be far more *natural*. **Use case:** Generating an SQL query with a list of columns and conditions can now be done inside a single f-string that spans multiple lines with comments, improving clarity. In Python 3.11, one might avoid f-strings for such tasks or break the expression into smaller pieces because multi-line expressions or certain characters were disallowed. Now, a developer can write:

```
query = f"SELECT {'', '.join(columns)} FROM {table} WHERE {" AND ".join([
    f"{col} > {threshold}" # filter each column by threshold
    for col, threshold in conditions
])}"
```

Such code is valid in 3.12 (even though it nests quotes and uses a list comprehension with comments inside the f-string) ¹⁷. The result is fewer concatenations or format calls, and code that directly represents the intended string structure. This improves **maintainability** because the string template and code are interwoven in one literal, which is easier to read and modify. The enhanced error messages for f-strings also mean when a mistake is made in these complex literals, the developer gets a pinpointed syntax error ²⁰, speeding up debugging.

- **Performance Gains in Critical Paths:** For developers working on performance-sensitive applications – such as data analysis, web services, or scientific computing – Python 3.12 offers modest speedups that can accumulate in long-running tasks. While a ~5% average speedup ⁴⁴ might not be noticeable in a short script, in large-scale jobs (e.g. a web server handling millions of requests, or a data pipeline processing gigabytes of data) it can translate to tangible resource savings. **Use case:** A web framework that performs many `isinstance()` checks against protocol classes (perhaps for validating whether objects conform to certain interfaces) will see these checks run much faster in 3.12 – possibly a 2–20× improvement ⁵³. This reduces overhead in request processing. Similarly, projects that make heavy use of comprehensions will benefit from the comprehension inlining (PEP 709), which could notably improve sections of code that construct large lists or dicts in tight loops ⁴⁵. The faster regex substitutions ⁵⁰ help text processing tasks, such as log parsing or data cleansing scripts, run quicker. For scientific computing, the more accurate `sum()` (with Neumaier summation) ⁴² means fewer floating-point error accumulations, which is a subtle but important improvement for numerical reliability.
- **Enhanced Concurrency and Profiling:** Although still needing C extensions or future Python APIs to fully leverage, the per-interpreter GIL (PEP 684) is a sign of Python's evolving story for concurrency. In specialized environments where Python is embedded (for instance, inside a database or a web server plugin system), it's now feasible to run independent Python interpreters truly in parallel ²⁴.

While typical developers won't use this feature directly in 3.12, its presence indicates that Python is moving toward better multi-core utilization – something that could benefit CPU-bound workloads down the line. For now, the **Linux perf integration** is immediately useful: developers profiling Python applications on Linux using the `perf` tool will see Python function names in the profiler output ¹⁰². This dramatically improves the usefulness of system-level profiling for Python code (previously, `perf` would show a lot of time in the interpreter with no visibility into which Python functions were consuming time). Combining this with the low-impact monitoring API (PEP 669), developers and researchers can instrument Python programs in production or testing with minimal overhead, obtaining fine-grained performance data or coverage information ²⁷. This facilitates performance tuning and debugging on large codebases, making Python more amenable to high-performance use cases.

- **Better Error Diagnosis for Education:** Python's improved error messages (a continuation in 3.12 of efforts started in 3.10) are particularly beneficial for **beginners and educators**. When a student or new programmer forgets an import or misnames an attribute, the interpreter's suggestions (e.g. *"NameError: name 'os' is not defined. Did you forget to import 'os'?"* ³⁴) can guide them to the solution quickly. This reduces frustration and teaching overhead, as the language itself provides hints. In a classroom or tutorial setting, using Python 3.12 means learners get immediate, helpful feedback for common mistakes (like mixing up `=` and `==`, or wrong import statements), aligning with Python's overall goal of being an accessible language.
- **Impact of Removed Features on Projects:** From a practical standpoint, the removal of `distutils` and other old modules means that some projects will need to update their dependencies. For example, a few legacy build scripts or packages might have imported `distutils` directly. In Python 3.12, these will break, so project maintainers should ensure they rely on `setuptools` or `build` tools that are up-to-date (the Python packaging ecosystem has largely moved on, so this impact is limited). Similarly, any code still using `asyncore` / `asynchat` (perhaps in custom network servers) will have to migrate to modern alternatives (such as `asyncio` or third-party async libraries) ⁶². In practice, the number of affected projects is small, given how old those modules are. The benefit for developers is that the standard library is now a bit leaner and focused on one way to do things (e.g., `asyncio` for async networking, `importlib` for imports, etc.), which can reduce confusion.

Overall, Python 3.12's changes were designed with real-world development in mind. The **practical impact** can be summarized as: a smoother development experience (through better errors and typing tools), slight performance improvements "for free" after upgrading, and the ability to use new stdlib conveniences instead of reinventing them. Professional developers will appreciate the incremental speed gains and cleaner APIs, researchers can build more efficient profiling and monitoring into their tools, and students will find the language more forgiving and instructive. As with any new Python version, adopting 3.12 should be preceded by running test suites to catch any use of removed features or any subtle behavior changes (the Python 3.12 release notes and porting guide enumerate these), but the transition from 3.11 to 3.12 is expected to be straightforward for most projects.

Comparisons to Python 3.11 and Earlier Versions

Comparing Python 3.12 to its immediate predecessor (3.11) and earlier releases helps contextualize the evolution of the language. Python 3.11 was a landmark release particularly for performance and error handling improvements, and Python 3.12 builds upon those foundations. Here are key comparative points:

- **Performance Trajectory:** Python 3.11 introduced the adaptive specializing bytecode interpreter (PEP 659), yielding dramatic speedups – in some cases, **10–60% faster execution** compared to Python 3.10 ¹⁰³. Python 3.12 did not seek to duplicate such a large leap, but rather to refine and extend these optimizations. As noted, 3.12's optimizations contribute roughly a further 5% average improvement over 3.11 ⁴⁴. In other words, if one's code ran in 1.0 seconds on 3.10, it might have dropped to ~0.7–0.9 seconds on 3.11, and with 3.12 it could be ~0.67–0.85 seconds (depending on workload). This continued (though smaller) improvement shows the Python core team's commitment to performance. It's also worth comparing startup times: Python 3.11 already improved startup by freezing some core modules, and 3.12 continues tweaks in that area (e.g., more efficient loading of certain stdlib modules) ¹⁰⁴, though no major startup overhaul occurred in 3.12.
- **Typing Enhancements:** Python 3.10 and 3.11 made notable strides in typing – e.g., Python 3.11 added new type constructors like `typing.Self`, `typing.LiteralString`, and support for variadic generics via `TypeVarTuple` (PEP 646) and `dataclass` transformations. However, using variadic generics in 3.11 still required clunky syntax. Python 3.12's PEP 695 can be seen as a direct answer to make **variadic and parametric generics more user-friendly**. In effect, Python 3.12 surpasses 3.11 by allowing things that were conceptually in 3.11 but not ergonomically accessible – for example, defining a generic class with a `*Ts` `TypeVarTuple` in 3.11 was possible but cumbersome, whereas 3.12 lets you write `class MyClass[*Ts]: ...` explicitly. The override decorator (PEP 698) in 3.12 also complements the typing story of 3.11, which had focused on new type primitives; 3.12 focuses on developer intent and error prevention in inheritance hierarchies ⁸⁴.
- **Error Handling and Messages:** Python 3.11 introduced Exception Groups and `except*` syntax (PEP 654) to handle multiple exceptions, which was a major new feature for error handling in concurrent programs. Python 3.12 did not introduce new syntax for exceptions, but it **polished error reporting**. As described, error messages in 3.12 are even more informative than those in 3.11, catching more instances of likely typos. The trend from 3.10 → 3.11 → 3.12 has been steadily improving feedback: 3.10 started the “Did you mean ...” suggestions for certain errors, 3.11 improved traceback detail with location of errors in expressions (PEP 657) and more suggestions, and 3.12 adds suggestions for imports and attributes ³⁴ ³⁵. These incremental improvements mean that newcomers transitioning from Python 3.9 or 3.10 to 3.12 will find a noticeably friendlier interpreter when mistakes are made. On the debugging side, Python 3.12's monitoring API (PEP 669) complements Python 3.11's rich traceback info – both aim to assist developers in diagnosing issues, albeit in different ways (one at runtime tooling level, the other in error output).
- **Asynchronous Features:** Python 3.11 expanded the asynchronous features by adding `taskgroups` in `asyncio` and improving error handling for tasks. Python 3.12's async improvements are more performance- and API-oriented: better socket write performance, new watcher defaults, and so on ⁵⁴ ⁸⁷. One specific comparison: Python 3.11 and 3.12 both support *asynchronous comprehensions*, but 3.12 optimizes them further (via comprehension inlining even in async contexts) ¹⁰⁵. The Ultimate Systems blog notes that Python 3.12 allows using async

comprehensions inside async functions more seamlessly ¹⁰⁶ (though this was largely possible in 3.11 as well). In effect, 3.12 continues refining the async paradigm introduced in 3.5 and boosted in 3.11, aiming for both correctness and speed in `asyncio`.

- **Pattern Matching and Syntax:** Structural pattern matching (PEP 634) was introduced in Python 3.10 and slightly refined in 3.11. Python 3.12 does not introduce new pattern matching syntax, but according to comparative discussions, it *permits more complex patterns* – possibly referring to support for nested patterns or subtle grammar loosening. The blog comparison suggests 3.12 handles nested patterns more flexibly ¹⁰⁷ ¹⁰⁸, though the official documentation doesn't call out a specific PEP for pattern matching in 3.12. It may be that improvements were minor or internal. Nonetheless, from Python 3.9 to 3.12, pattern matching went from non-existent to a mature feature, and 3.12's primary syntax changes were elsewhere (typing and f-strings).
- **“Dead Battery” Removal Progress:** Python 3.11 began the process of sunsetting old modules (it issued deprecation warnings for some, like `smtpd` and `distutils`). Python 3.12 is where these deprecations took effect as removals ⁶¹. So, compared to 3.11, Python 3.12 has a smaller standard library surface (by dropping several modules). From the perspective of someone upgrading from 3.10 or 3.11, this means a few more breaking changes: code that emitted only a warning in 3.11 now fails in 3.12 if not updated. However, historically, the shift from 3.x to 3.(x+1) has rarely removed so many modules at once – this is a notable housekeeping milestone. The removal of `distutils` in 3.12 is analogous to how Python 3.8 removed `asyncio.coroutines` (legacy decorator) or Python 3.9 removed `parser` module: it's the culmination of a deprecation plan. Future versions (3.13+) will likely continue this (PEP 594 schedules more removals through 3.13 and 3.14 for other old modules like `cgi`, `mailbox`, etc., some of which may still be pending).

In summary, **Python 3.12 vs 3.11** can be seen as an iteration that refines the big changes of 3.11 rather than introducing radical new concepts. Python 3.11 was focused on speed and better errors; Python 3.12 makes speedups more concrete in specific areas (with new optimizations like PEP 709 and BOLT) and further enhances error/help infrastructure. It also adds a major typing improvement (PEP 695) that had no equivalent in 3.11, showing Python's increasing emphasis on static typing support. Compared to older versions like 3.9 or 3.10, Python 3.12 will feel significantly faster and more helpful, due to the cumulative effect of changes since 3.10 (pattern matching, interpreter optimizations, improved tracebacks, etc., on top of 3.12's own new features). Thus, Python 3.12 represents a step forward but in continuity with the direction set by its immediate predecessors, emphasizing a balance of **usability and performance** improvements.

Conclusion

Python 3.12 stands as a comprehensive enhancement of the Python language, blending **innovative new features** with **incremental improvements** that collectively enhance the development experience. On the language front, the introduction of a more compact generics syntax (PEP 695) and full-fledged f-string grammar (PEP 701) empowers developers to write clearer, more expressive code with less boilerplate and fewer pitfalls ⁹ ¹⁵. Under the hood, interpreter-level advancements like the per-interpreter GIL (PEP 684) and low-impact monitoring (PEP 669) reflect ongoing efforts to modernize Python's runtime for better concurrency and tooling support ²⁴ ²⁷. Performance-wise, Python 3.12 continues the momentum of its predecessor by optimizing common operations and reducing overheads – while the ~5% overall speedup ⁴⁴ may seem modest, it comes on top of Python 3.11's significant gains and is achieved through careful tuning (comprehension inlining, improved bytecode, faster built-ins, etc.) with broad applicability.

The standard library in 3.12 has been refreshed: new conveniences (like `itertools.batched()` and `pathlib.Path.walk()`) reduce the need for third-party utilities ⁸⁰ ¹⁰⁹, and deprecated modules have been retired, keeping the language lean and focused ⁶¹. These changes serve both the present and the future – by removing legacy baggage, Python can evolve more freely, and by adding new tools, developers can be more productive out-of-the-box.

For students and newcomers, Python 3.12 is more welcoming than ever, with interpreter messages that guide them to solutions and a syntax that stays out of the way of understanding (e.g., no mysterious f-string limitations to trip over). For professional developers, 3.12 offers practical benefits: type checking in large codebases becomes more robust and convenient, and existing code potentially runs faster without modification. Researchers and advanced users gain new capabilities for introspection and parallelism that can be the foundation for future innovations (for instance, experimental multi-core processing models or more precise profilers).

In comparison to earlier Python versions, 3.12 might be characterized as a *refinement release* – it does not have a single headline feature like “pattern matching” (3.10) or “massive speedups” (3.11) that fundamentally changes how one writes Python, but it smooths many rough edges and pushes the language forward on multiple fronts simultaneously. It continues Python’s tradition of maintaining backward compatibility reasonably well (most breaking changes were signaled far in advance) while still taking bold steps such as removing `distutils` and other old stdlib components for the greater good of the ecosystem ⁶¹.

Looking ahead, the developments in Python 3.12 also lay groundwork for future releases: the per-interpreter GIL hints at a future where Python might effectively utilize multiple cores (perhaps foreshadowing the potential acceptance of proposals like PEP 703 to make the GIL optional), and the internal changes (immortal objects, integer representation) pave the way for further performance and memory optimizations in Python 3.13 and beyond. Meanwhile, the success of PEP 695 in simplifying generics could lead to even more powerful typing features (e.g., default type parameters, as proposed in PEP 696).

In conclusion, Python 3.12 is a testament to the language’s evolution, balancing new capabilities with thoughtful cleanup. It demonstrates how a mature language can continue to improve developer productivity and program performance hand-in-hand. For the Python community – whether in academia, industry, or hobbyist circles – adopting Python 3.12 should be a largely positive experience, bringing tangible benefits with relatively low migration effort. This release reinforces Python’s status as a **versatile, high-level language** that does not rest on its laurels but incrementally moves forward, guided by PEPs and community input to meet the growing needs of its diverse user base.

Sources:

- Python Software Foundation. *What’s New In Python 3.12*. Python Documentation, Oct. 2023 ² ⁹ ¹⁵ ²⁴.
- Python Release Team. *Python 3.12.0 Release Highlights*. Python.org, Oct. 2023 ⁴ ³ ⁶³.
- Galindo, Pablo, et al. *PEP 701 – Syntactic Formalization of F-Strings*. Python Enhancement Proposals, 2023 ¹¹⁰ ²¹.
- Traut, Eric, et al. *PEP 695 – Type Parameter Syntax*. Python Enhancement Proposals, 2023 ⁷ ⁸.

- Shannon, Mark, et al. *PEP 669 – Low Impact Monitoring for CPython*. Python Enhancement Proposals, 2022 ²⁷ .
- Snow, Eric. *PEP 684 – A Per-Interpreter GIL*. Python Enhancement Proposals, 2022 ²³ .
- Python Core Developers. *CPython 3.12 Change Log and Commit History*. (Extracted performance and module changes) ⁵⁰ ¹¹¹ .
- Ultimate Systems. "Python 3.11 vs Python 3.12: Difference Comparison." *Ultimate Systems Tech Blog*, Nov. 2024 ¹⁰³ ¹⁰⁶ .
- Geir Arne Hjelle. "Python 3.12: Cool New Features for You to Try." *Real Python*, Sep. 2023 ¹¹² ⁸⁹ .

1 2 5 6 7 8 9 10 11 12 14 15 16 17 18 19 20 23 24 25 26 27 28 29 30 31 32 33 34 35
 36 37 39 40 41 42 45 46 47 48 49 50 51 52 53 54 55 56 57 58 60 62 64 65 66 67 68 69 70 71
 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100 101

¹⁰⁹ ¹¹¹ What's New In Python 3.12 — Python 3.13.5 documentation

<https://docs.python.org/3/whatsnew/3.12.html>

³ ⁴ ¹³ ³⁸ ⁴⁴ ⁵⁹ ⁶¹ ⁶³ ¹⁰² Python Release Python 3.12.0 | Python.org

<https://www.python.org/downloads/release/python-3120/>

²¹ ²² ¹¹⁰ PEP 701 – Syntactic formalization of f-strings | peps.python.org

<https://peps.python.org/pep-0701/>

⁴³ ¹⁰³ ¹⁰⁴ ¹⁰⁵ ¹⁰⁶ ¹⁰⁷ ¹⁰⁸ Python 3.11 vs. Python 3.12: Difference Comparison – Ultimate Systems Blog

<https://blog.usro.net/2024/11/python-3-11-vs-python-3-12-difference-comparison/>

¹¹² Python 3.12: Cool New Features for You to Try – Real Python

<https://realpython.com/python312-new-features/>